GATEFLOW – INTELLIGENT CAMPUS ACCESS AND VISITOR AUTHORIZATION PLATFORM

A PROJECT REPORT

Submitted by

SHRUTI K

(2116240701510)

in partial fulfilment of the End Semester Examination for the course

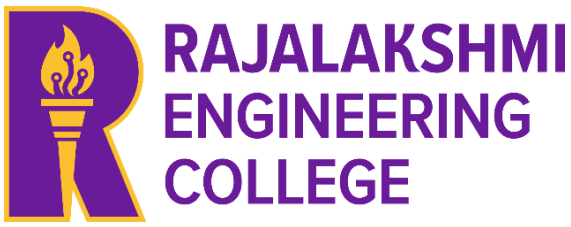
CB23F35

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF ENGINEERING

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "gateflow – Intelligent Campus Access and Visitor Authorization Platform. System" is the bonafide work of SHRUTI K (2116240701510), who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester V, under my supervision.

SIGNATURE

Dr. J. Manoranjini
HEAD OF THE DEPARTMENT
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

Dr. J. Jinu Sophia
COURSE FACULTY
Associate Professor
Department of Computer Science and Engineering
Rajalakshmi Engineering College

Submitted for the End Semester Examination (Individual Project) held on November 2026.

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, SHRUTI K (2116240701510), hereby declare that the project report titled "GateFlow – Intelligent Campus Access and Visitor Authorization Platform", submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, pages, Deluge scripts, workflows and integrations — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report.

Place: Chennai
Date: 11.11.2026

SIGNATURE OF THE STUDENT
SHRUTI K

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman, Mr. S. Meganathan, B.E., F.I.E., Chairperson, Dr. Thangam Meganathan, Ph.D., and Vice-Chairman, Mr. Abhay Shankar Meganathan, B.E., M.S., for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, Dr. S. N. Murugesan, M.E., Ph.D., Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank Dr. J. Manoranjini, Professor and Head of the Department, Department of Computer Science and Engineering, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, Dr. J. Jinu Sophia, Associate Professor, for the valuable guidance, reviews, and feedback that shaped this application and report.

I also acknowledge Zoho Corporation and the Zoho Creator Education Programme for providing access to the Zoho Creator platform and learning resources. Finally, I thank my family and friends for their unwavering support.

SHRUTI K

ABSTRACT

Offices and college campuses traditionally maintain visitor records using paper registers at security desks. This manual process involves recording visitor details, contacting employees for approval and tracking entry and exit times. It often leads to delays, incomplete records and difficulties in monitoring the number of visitors present on the premises.

GateFlow – Intelligent Campus Access and Visitor Authorization Platform is a Zoho Creator application developed to automate and simplify visitor management. It consists of three interconnected forms – Sites, Employees and Visits – linked through lookup fields. Visitors can submit visit requests in advance, which are validated and saved with a Pending status. Deluge workflows automatically send an acknowledgement email to the visitor and an approval notification to the respective host.

Hosts can approve or reject requests through the Pending Approvals report, while security personnel manage visitor check-in and check-out through report actions. A central Deluge function, Process visit, handles status transitions and validates user roles, visit dates, record status and the maximum visitor capacity of each site. The application also provides Admin, Host and Security roles, five list reports and a dashboard with four key counters for monitoring visitor activities.

The application was tested in the Development environment using synthetic records. Tests confirmed that invalid visit dates, incorrect arrival and departure sequences, and zero site capacity were rejected. The system also prevented entry for pending requests and blocked check-ins when the site reached its visitor capacity. Available capacity was restored after check-out, and dashboard counts matched the stored records. Visitor acknowledgement and host notification emails were successfully tested. Separate role-based login and simultaneous check-in verification remain pending, while optional REST webhook integration was not included in this implementation.

Keywords: *Low-Code Development; Zoho Creator; Deluge Scripting; Workflow Automation; Role-Based Access Control; Visitor Management*

PROJECT AT A GLANCE

Attribute	Details
Project Title	GATEFLOW – Intelligent Campus Access and Visitor Authorization Platform.
Application Domain	Facility security – visitor request, approval and entry/exit management
Student Name & Register No.	SHRUTI K (2116240701510)
Zoho Creator Edition	Trial account (zoho.in data centre)
Application Name (Link Name)	GATEFLOW – Intelligent Campus Access and Visitor Authorization Platform.
Published / Live URL	https://shrutikumaran24161.zohocreatorportal.in
Demo Credentials (per role)	Admin: owner account | Host: GateFlow Host profile | Security: GateFlow Security profile
Number of Forms / Subforms	3 / 0
Number of Reports (by type)	List: 5 Kanban: 0 Calendar: 0 Pivot/Chart: 0 Others: 0
Number of Pages / Dashboards	1 (GateFlow_Dashboard)
Number of Workflows	Form: 5 Schedule: 0 Approval: 0 Blueprint: 0 Batch: 0 (plus 4 report workflows)
Deluge Functions / Custom Actions	2 functions (ProcessVisit, VisibleVisitIDs) / 4 report actions
Integrations	Email notifications through Deluge sendmail (Zoho Mail); REST webhook not implemented
Roles & Profiles Defined	Roles: Admin, Host, Security; Profiles: GateFlow Host, GateFlow Security
Environments Used	Development only (Stage and Production not used)
Date of Production Deployment	Not deployed to Production

TABLE OF CONTENTS

ABSTRACT iv
PROJECT AT A GLANCE v
LIST OF TABLES viii
LIST OF FIGURES ix
LIST OF ABBREVIATIONS x
INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 1
1.4 Scope of the Project 2
1.5 Justification for a Low-Code Approach 2
1.6 Organisation of the Report 2
LITERATURE SURVEY AND SYSTEM STUDY 3
2.1 Study of the Existing System 3
2.2 Limitations of the Existing System 3
2.3 Low-Code versus Traditional Development 4
2.4 Literature Survey 4
2.5 Emerging Trends in Low-Code Development 5
2.6 Proposed System 5
REQUIREMENT ANALYSIS 6
3.1 Stakeholders and User Roles 6
3.2 Functional Requirements 6
3.3 Non-Functional Requirements 7
3.4 Hardware and Software Requirements 7
3.5 Use Case Model 8
SYSTEM DESIGN 9
4.1 System Architecture 9
4.2 Data Model 10
4.3 Form Design 10
4.4 Process and Workflow Design 12
4.5 User Interface and Navigation Design 13
4.6 Access Control Design 13
IMPLEMENTATION 14
5.1 Application Setup 14
5.2 Forms and Data Capture 14
5.3 Reports 15
5.4 Pages and Dashboards 16
5.5 Deluge Scripting 17
5.6 Workflows and Automation 18

5.7 Integrations 19
5.8 User Management and Role-Based Access 20
5.9 AI and Analytics Features 21
5.10 Pro-Code Extension 21
5.11 Mobile and Portal Access 21
TESTING AND DEBUGGING 22
6.1 Testing Strategy 22
6.2 Test Cases 22
6.3 Debugging Deluge Scripts 23
6.4 Defect Log 24
6.5 User Acceptance Testing 24
DEPLOYMENT, SECURITY AND GOVERNANCE 25
7.1 Environments and Deployment Strategy 25
7.2 Performance Optimisation 25
7.3 Security Features 26
7.4 Governance and Administration 26
RESULTS AND DISCUSSION 27
8.1 Application Output 27
8.2 Achievement of Objectives 28
8.3 Comparison with the Existing System 29
8.4 Limitations of the Low-Code Approach 29
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum 29
CONCLUSION AND FUTURE ENHANCEMENTS 30
9.1 Conclusion 30
9.2 Future Enhancements 30
9.3 Learning Outcomes and Career Relevance 30
REFERENCES 31
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST 32
APPENDIX B – COURSE OUTCOME MAPPING 34
APPENDIX C – COMPLETE DELUGE SOURCE CODE 35
APPENDIX D – USER MANUAL 39
D.1 Accessing the Application 39
D.2 Instructions for Host 39
D.3 Instructions for Security 39

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application 4
Table 2.2: Summary of Literature Survey 4
Table 3.1: Stakeholders and User Roles 6
Table 3.2: Functional Requirements 6
Table 3.3: Non-Functional Requirements 7
Table 3.4: Hardware and Software Requirements 7
Table 3.5: Use Case Description – Approve or Reject a Visit Request 8
Table 4.1: Entities and Relationships 10
Table 4.2: Field Specification – Sites Form 10
Table 4.3: Field Specification – Employees Form 11
Table 4.4: Field Specification – Visits Form 11
Table 4.5: Role–Permission Matrix 13
Table 5.1: Reports Implemented 15
Table 5.2: Deluge Scripts Implemented 17
Table 5.3: Workflows Implemented 18
Table 5.4: Integrations Implemented 19
Table 6.1: Test Cases 22
Table 6.2: Defect Log 24
Table 6.3: User Acceptance Feedback 24
Table 7.1: Deployment Checklist 25
Table 7.2: Performance Optimisation Results 26
Table 7.3: Security Controls Implemented 26
Table 8.1: Mapping of Objectives to Outcomes 28
Table 8.2: Existing System versus Proposed Application 29
Table A.1: Feature Coverage Checklist 32
Table B.1: Mapping of Course Outcomes to Report Chapters 34

LIST OF FIGURES

Figure 2.1: As-Is Process Flow of the Existing Visitor Register 3
Figure 3.1: Use Case Diagram of GateFlow 8
Figure 4.1: System Architecture of GateFlow 9
Figure 4.2: Entity–Relationship Diagram of GateFlow 10
Figure 4.3: Workflow / Process Flow for a Visit Request 12
Figure 4.4: Navigation Structure and Wireframe of the GateFlow Dashboard 13
Figure 5.1: Application Home and Navigation Sections 14
Figure 5.2: Visits Form in the Form Builder 15
Figure 5.3: Visits Form in Live Mode (Host disabled until a Site is selected) 15
Figure 5.4: All Visits Report with Synthetic Test Records 16
Figure 5.5: GateFlow Dashboard Page (viewed on 01 November 2026) 17
Figure 5.6: Form Workflows Configured on the Visits Form 19
Figure 5.7: Notify Visitor and Host Workflow (sendmail configuration) 20
Figure 5.8: Roles and Permission Profiles Configuration (GateFlow Host and GateFlow Security) 21
Figure 5.9: GateFlow in the Zoho Creator Tablet Layout Preview 21
Figure 6.1: ProcessVisit Function in the Deluge Editor (Save / Execute) 23
Figure 7.1: Environments Page (Stage and Production not yet used) 25
Figure 7.2: Audit Trail of the Visits Form 26
Figure 8.1: Scenario Step 1 – Visit Request entered on the Visits Form 27
Figure 8.2: Scenario Step 2 – Approved and Checked-In Visit with Approval Note 28
Figure 8.3: Scenario Step 3 – Check Out Action in the Active Visitors Report 28

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
ER	Entity–Relationship
HTML	HyperText Markup Language
KPI	Key Performance Indicator
LCAP	Low-Code Application Platform
OAuth	Open Authorization
PII	Personally Identifiable Information
RBAC	Role-Based Access Control
REST	Representational State Transfer
SDLC	Software Development Life Cycle
UAT	User Acceptance Testing
UI	User Interface
Zia	Zoho's AI assistant

CHAPTER 1
INTRODUCTION

1.1 Background

Every office building, college campus and industrial site receives a steady stream of people who do not work there: candidates coming for interviews, vendors, delivery staff, maintenance technicians, parents and guests attending meetings. The security desk at the gate is responsible for letting these visitors in, making sure that someone inside is actually expecting them, and knowing who is still on the premises at the end of the day.

The project considers a realistic campus with one or more sites (for example a main block and an annexe). Each site has its own security staff and a maximum number of visitors that can be inside at once. Employees act as hosts, and the administration office maintains the list of sites and employees. GateFlow digitises this process on Zoho Creator so that a visit is requested and approved before the visitor reaches the gate, and the gate only has to check the status and record entry and exit.

1.2 Problem Statement

The existing paper-based visitor register gives no structured way to request a visit in advance, obtain and record the host's approval, enforce the visitor capacity of a site or know who is currently inside, which causes delays at the gate and leaves the institution without reliable entry and exit records.

- Approval depends on the guard reaching the host by phone, so visitors wait at the gate and the decision is not recorded.
- Entry and exit times are written by hand; exit entries are often missed, so the number of people inside is unknown.
- There is no check against the capacity of a site, and nothing stops a visitor from being let in on the wrong date.
- Anyone at the desk can see or alter entries, and old pages are slow to search during an audit.

1.3 Objectives

- To design a normalised data model and forms for sites, employees and visits, linked through lookup fields.

- To automate request validation, host filtering, status transitions and capacity control using Deluge scripting and workflows.
- To provide role-based dashboards and reports for pending approvals, active visitors and the complete visit history.
- To integrate the application with Zoho Mail through Deluge sendmail for acknowledgement, approval-request and decision emails.
- To secure the application with roles, permission profiles and record-level filters so that each user sees and acts only on permitted visits.

1.4 Scope of the Project

In scope. Sites, Employees and Visits forms; request validation; host filtering by site; email notifications; approval by the assigned host; check-in and check-out with a capacity check; reports for each stage; one dashboard page; and Admin, Host and Security roles, used through the Creator web and mobile layouts.

Out of scope. As required by the assignment, QR passes, a visitor self-service portal, identity document uploads, facial recognition, SMS or WhatsApp messages, separate audit or notification forms and a separate front end were not built. The optional REST webhook on check-in was also left out of this build.

1.5 Justification for a Low-Code Approach

A visitor management system is mostly made up of forms, lookups, status changes, notifications and role-based views. These are exactly the building blocks that a low-code platform provides ready-made [1]. On Zoho Creator the database, user login, hosting, web and mobile interfaces and email service are already available, so the development effort goes into the business rules rather than into infrastructure [2].

1.6 Organisation of the Report

Chapter 2 reviews the existing system and related low-code literature. Chapter 3 presents the requirement analysis. Chapter 4 describes the system design. Chapter 5 details the implementation on Zoho Creator. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, performance optimisation, security and governance. Chapter 8 presents results and discussion, and Chapter 9 concludes the report with future enhancements.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

The existing process was studied as it is commonly followed at a campus security desk. A visitor arrives without prior notice and gives his or her name and the person to be met. The guard writes these details in a bound register and phones the host. Once the host confirms, the visitor is allowed in and the entry time is noted. When the visitor leaves, the guard is expected to write the exit time against the same entry. At the end of the day the register is kept in a cupboard and is consulted only when a question arises. Figure 2.1 shows this flow and the problems observed at each stage.

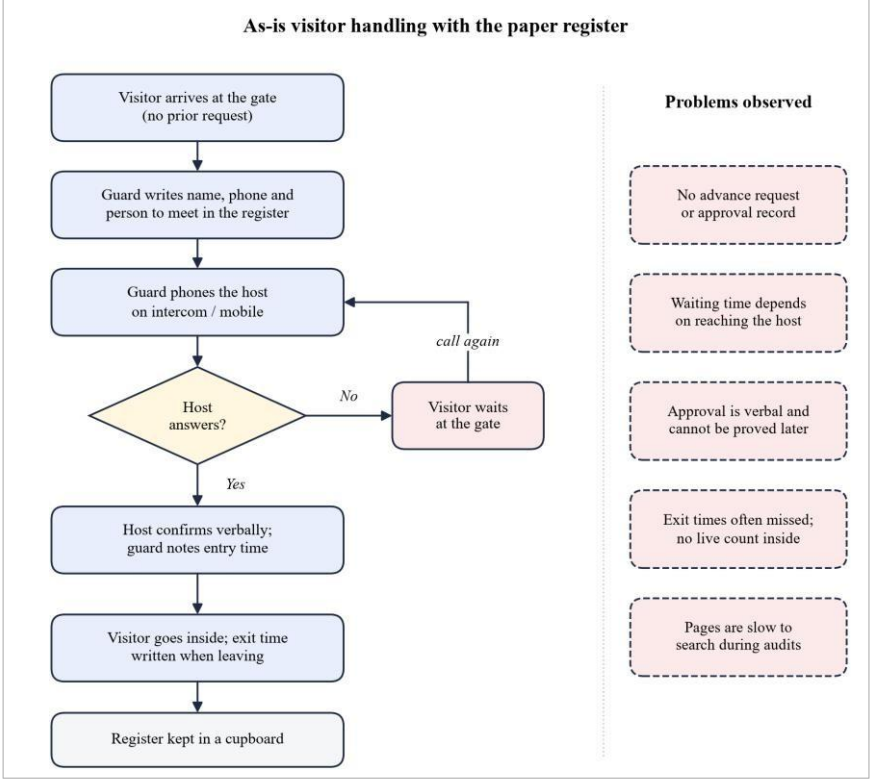


Figure 2.1: As-Is Process Flow of the Existing Visitor Register

2.2 Limitations of the Existing System

- No advance request: the host learns about the visit only when the visitor is already at the gate.
- Approvals are verbal and are not stored, so it cannot be proved later who allowed a visitor in.
- There is no live count of visitors inside a site and no way to enforce a maximum capacity.

- Exit times are frequently missing, so the register cannot be trusted for safety or evacuation purposes.
- Records are hard to search, and any person at the desk can view or change entries.

2.3 Low-Code versus Traditional Development

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Development time	About 6–8 weeks (estimate)	About 1–2 weeks (estimate)
Skills required	Front end, back end, SQL, deployment	Creator builders and basic Deluge
Hosting & infrastructure	Own server, database, SSL, backups	Hosted by Zoho
Mobile application	Separate responsive or native build	Creator mobile app and layouts
Security & access control	Login, roles and sessions hand-coded	Built-in login, roles, profiles, PII flags
Maintenance & change	Code, test and redeploy	Edit in builder; live immediately
Customisation limits	Practically unlimited	Creator components; HTML snippet for dashboard
Cost	Developer time plus hosting	Per-user subscription (trial used)

2.4 Literature Survey

Table 2.2: Summary of Literature Survey

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
1	W. Rosam, O. Kalnichevski [1]	Low-Code Development: A Practical Guide	Visual builders and platform services shorten delivery of business apps.	Basis for the low-code approach.
2	Zoho Corporation [2], [3]	Creator Help; Deluge Reference	Forms, lookups, report actions, pages, profiles, Deluge tasks.	Used while building every component.
3	R. Sahay et al., 2020 [6]	Comparison of low-code platforms	Common features: data model, logic, roles, deployment.	Features the project must exercise.

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
4	A. C. Bock, U. Frank, 2021 [7]	Low-Code Platform	Benefits and limits such as lock-in and restricted customisation.	Basis for Section 8.4.
5	Gartner, Inc. [5]	Magic Quadrant for Enterprise LCAP	Growing use of low-code for departmental process apps.	Relevance of low-code skills.

2.5 Emerging Trends in Low-Code Development

Several current trends in low-code development are visible in this project. Platforms now include AI-assisted builders; Zoho Creator shows a Zia "Build Agent" panel inside the application builder that can generate forms from a prompt. For GateFlow this panel was not used: the forms, workflows and scripts were created manually so that every rule could be explained during evaluation [4].

Citizen development, where staff close to the problem build their own tools [8], is another trend; GateFlow keeps its rules readable for that reason. On a small scale it also follows hyper-automation by linking the request form, emails, report actions and dashboard into one paperless flow.

2.6 Proposed System

In the proposed Zoho Creator application a visit is requested in advance, the host approves or rejects it from a report after an email alert, and security checks the visitor in and out on the visit day. Each limitation listed in Section 2.2 is addressed as follows: the request is made in advance; the approval is stored with the approver and time; check-in counts the visitors already inside and stops at the site's capacity; check-in and check-out times are set by the system; reports can be searched and filtered; and roles restrict who can approve and who can record movement.

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Administrator	Application owner	Maintain sites and employees; view and process all visits
Host	Employee being visited	Approve or reject assigned requests
Security	Guard at a site	Check visitors in and out at that site
Visitor	External person (no login)	Give visit details; receive emails

3.2 Functional Requirements

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	Maintain sites with unique code, name, address, timezone, positive capacity and active flag.	Administrator	High
FR-02	Maintain employees with unique code and email, department, site, role and active flag.	Administrator	High
FR-03	Accept a visit request (visitor, site, host, purpose, date, times) and save it as Pending.	Visitor, Administrator	High
FR-04	Reject a past visit date and a departure not later than arrival.	All	High
FR-05	Offer only active sites and only active hosts of the selected site.	All	High
FR-06	Email an acknowledgement to the visitor and an approval request to the host on submission.	Visitor, Host	Medium
FR-07	Let only the assigned host or administrator approve/reject a Pending visit and email the decision.	Host, Administrator	High
FR-08	Allow check-in only for an Approved visit on its date while occupancy is below capacity.	Security, Administrator	High
FR-09	Allow check-out only for a checked-in visitor and record the time.	Security, Administrator	High
FR-10	Block direct edits of status and check-in/out times.	All	High
FR-11	Provide All Visits, Pending Approvals and Active Visitors reports filtered per user.	All roles	Medium
FR-12	Show a dashboard with today, pending, inside and completed counts.	All roles	Medium

FR ID	Requirement	Role(s)	Priority
FR-13	Send a minimal check-in event to an HTTPS webhook (no contact details).	System	Low (deferred)

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	Reports and the dashboard shall open within a few seconds for the daily visit volume of a campus.
NFR-02	Security	Users shall access only the data permitted by their role and profile; processing rules shall be checked on the server in Deluge.
NFR-03	Usability	The application shall be usable on the web and in the Zoho Creator mobile and tablet layouts.
NFR-04	Availability	The application shall be hosted on Zoho Creator so that no local server is required.
NFR-05	Maintainability	Status-changing rules shall be kept in one Deluge function so that they are changed in one place.
NFR-06	Auditability	Record additions and edits shall be traceable through the audit trail, and approvals shall store the approver and time.

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator (trial account, zoho.in data centre)
Scripting language	Deluge (Data Enriched Language for the Universal Grid Environment)
Browser	Google Chrome / Microsoft Edge / Mozilla Firefox (latest version)
API testing tool	Not used in this build (no outgoing REST call)
Other Zoho services	Zoho Mail (through Deluge sendmail)
Mobile	Zoho Creator mobile and tablet layout preview
Client hardware	Intel Core i3/i5 or above, 8 GB RAM, internet connection

3.5 Use Case Model

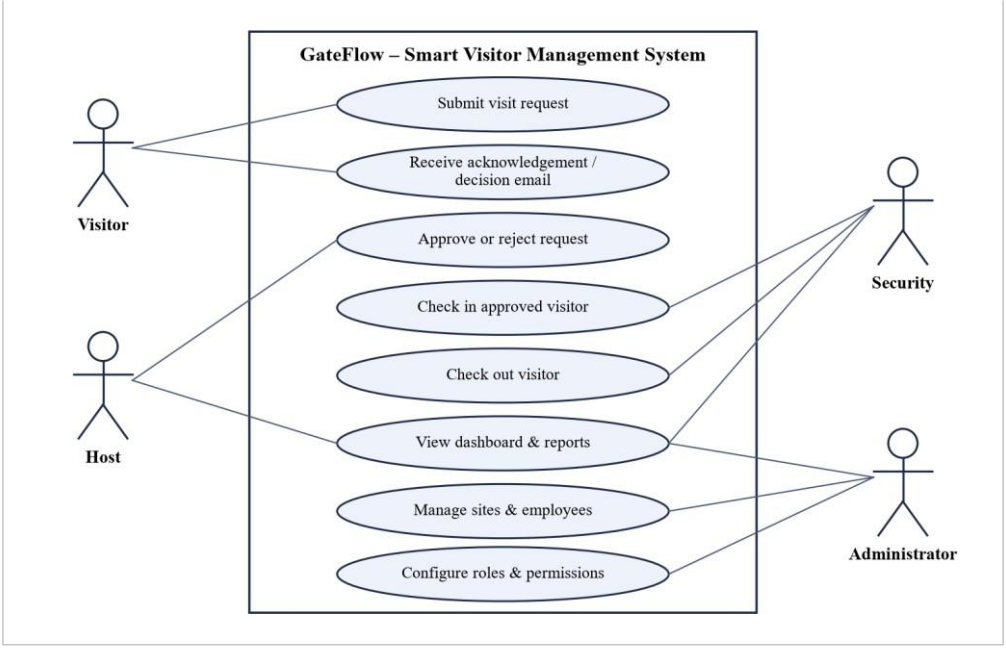


Figure 3.1: Use Case Diagram of GateFlow

Table 3.5: Use Case Description – Approve or Reject a Visit Request

Use Case ID	UC-01
Use Case Name	Approve or reject a visit request
Actor(s)	Host (Administrator may also act)
Pre-condition	The visit is Pending; the host is active and is the host selected on the request.
Main Flow	1. The host opens the Pending Approvals Report. 2. The host selects the request and clicks Approve or Reject. 3. ProcessVisit checks the user, the status, the date and the site and host. 4. The status and approval note are updated. 5. A decision email is sent to the visitor.
Alternate Flow	If the user is not the assigned host, or the visit is no longer Pending, or the date has passed, a message starting with “Blocked:” is returned and the record is not changed.
Post-condition	The visit is Approved or Rejected and leaves the Pending Approvals Report.
Related FR	FR-06, FR-07, FR-11

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

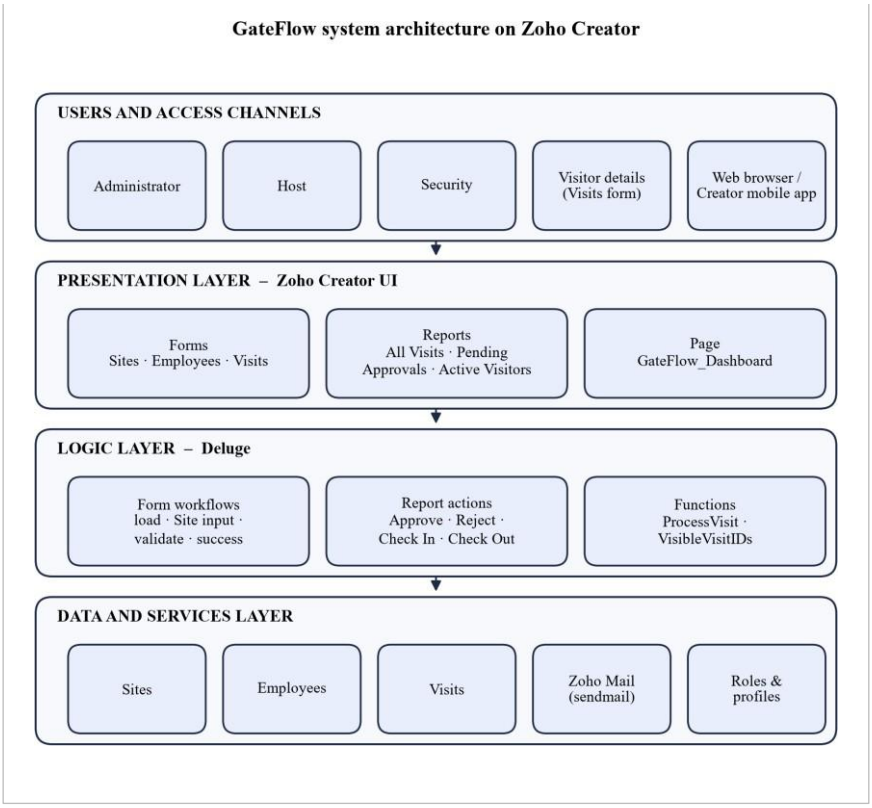


Figure 4.1: System Architecture of GateFlow

The presentation layer holds the three forms, the list reports and the dashboard page. The Deluge logic layer has form workflows (load, Site input, validate, success), four report workflows and two functions: ProcessVisit for every status change and VisibleVisitIDs for the report filters. The data layer is formed by the Sites, Employees and Visits forms, while email is sent through the platform's sendmail task. Roles and permission profiles sit across all layers and decide which forms, reports and actions each user can reach.

4.2 Data Model

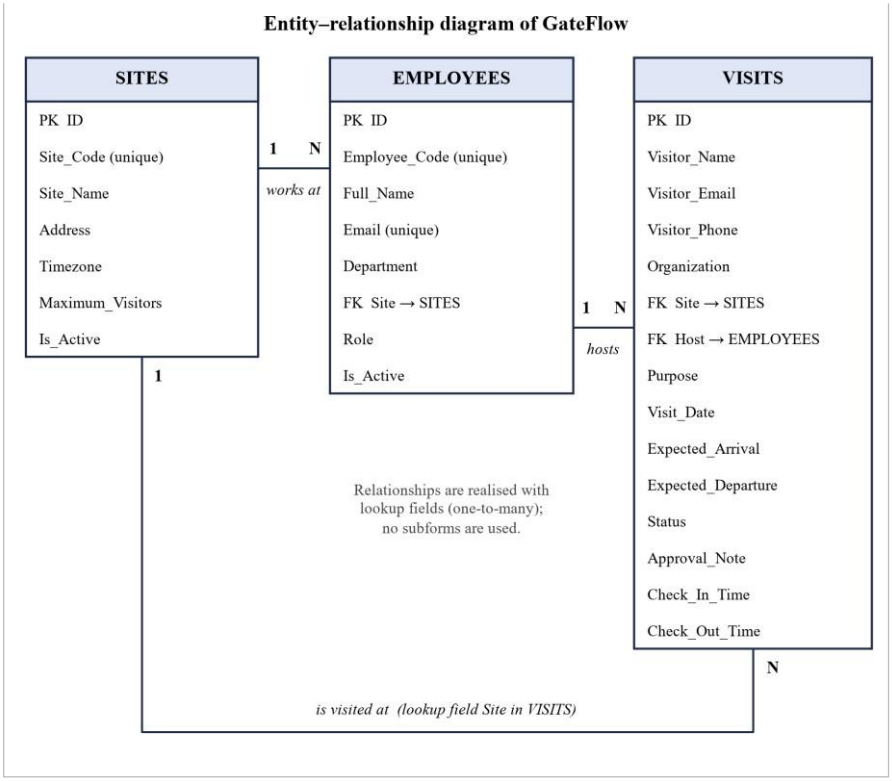


Figure 4.2: Entity–Relationship Diagram of GateFlow

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Sites	Site_Code, Site_Name, Maximum_Visitors, Is_Active	Employees	One-to-Many	Lookup Site in Employees
Sites	Site_Code, Site_Name, Maximum_Visitors, Is_Active	Visits	One-to-Many	Lookup Site in Visits
Employees	Employee_Code, Email, Role, Site, Is_Active	Visits	One-to-Many (host)	Lookup Host in Visits

4.3 Form Design

Table 4.2: Field Specification – Sites Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Site Code	Site_Code	Single Line	Yes	No duplicate values	Site identifier, e.g. SYN-01

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Site Name	Site_Name	Single Line	Yes	—	Display name
Address	Address	Address	No	—	Full address
Maximum Visitors	Maximum_Visitors	Number	Yes	Greater than 0	Capacity used at check-in
Timezone	Timezone	Single Line	Yes	Default: Asia/Kolkata	—
Is Active	Is_Active	Checkbox	—	Default: checked	Only active sites are offered

Table 4.3: Field Specification – Employees Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Full Name	Full_Name	Name	Yes	—	PII
Employee Code	Employee_Code	Single Line	Yes	No duplicate values	—
Email	Email	Email	Yes	No duplicate values	PII; login match
Role	Role	Drop Down	Yes	Default: Host	Host / Security / Admin
Site	Site	Lookup (Sites)	Yes	—	Site where the employee works
Is Active	Is_Active	Checkbox	—	Default: checked	Inactive staff cannot act
Department	Department	Drop Down	Yes	—	Six departments

Table 4.4: Field Specification – Visits Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Visitor Name	Visitor_Name	Name	Yes	—	PII; first and last name
Visitor Email	Visitor_Email	Email	Yes	Valid email format	PII; used for emails
Visitor Phone	Visitor_Phone	Phone	No	Default code +91	PII
Organization	Organization	Single Line	No	—	—
Site	Site	Lookup (Sites)	Yes	Active sites only (DS-02)	—
Host	Host	Lookup (Employees)	Yes	Hosts of site (DS-03)	Disabled until site chosen
Purpose	Purpose	Drop Down	Yes	—	Five purposes

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Visit Date	Visit_Date	Date	Yes	Not past; default today	—
Expected Arrival	Expected_Arrival	Time	Yes	—	—
Expected Departure	Expected_Departure	Time	Yes	After arrival (DS-04)	—
Status	Status	Drop Down	—	Default: Pending	Six status values
Approval Note	Approval_Note	Multi Line	No	Set by ProcessVisit	Hidden on request
Check In Time	Check_In_Time	Date-Time	—	Set by the system	Hidden on request
Check Out Time	Check_Out_Time	Date-Time	—	Set by the system	Hidden on request

4.4 Process and Workflow Design

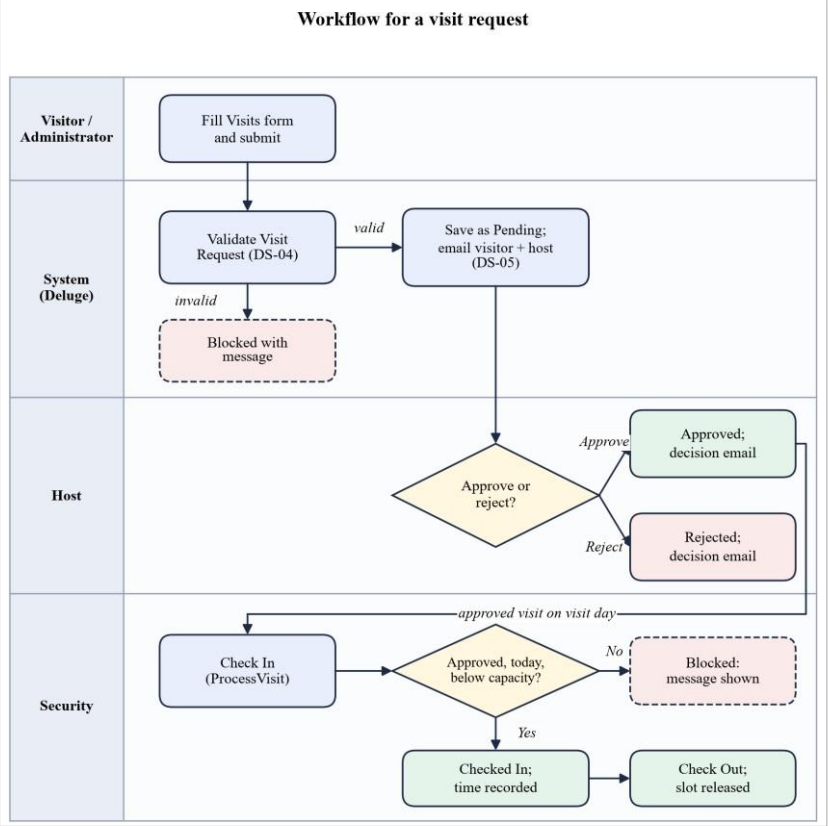


Figure 4.3: Workflow / Process Flow for a Visit Request

4.5 User Interface and Navigation Design



Figure 4.4: Navigation Structure and Wireframe of the GateFlow Dashboard

4.6 Access Control Design

Table 4.5: Role–Permission Matrix

Component	Administrator	Host	Security	Visitor
Forms: Sites, Employees	A E V D	—	—	—
Form: Visits	A E V D	V (assigned)	V (own site)	Request details only
Report: All Visits	V + all actions	—	V + Check In / Out	—
Report: Pending Approvals	V + Approve / Reject	V + Approve / Reject	—	—
Report: Active Visitors	V + Check Out	—	V + Check Out	—
Page: GateFlow Dashboard	V	V (own counts)	V (site counts)	—

CHAPTER 5
IMPLEMENTATION

5.1 Application Setup

The application was created from scratch (no template, import or AI builder) in the account shrutikumaran24@gmail.com with the link name gateflow-smart-visitor-management-system, using IST, the dd-MMM-yyyy date format and the 24-hour clock.

The left navigation menu is organised into four sections. The Sites section holds the Sites form and the All Sites report; the Employees section holds the Employees form and the All Employees report; the Visits section holds the Visits form and the All Visits, Pending Approvals and Active Visitors reports; and the last entry opens the GateFlow Dashboard page. The default dark theme of the new Creator UI was retained.

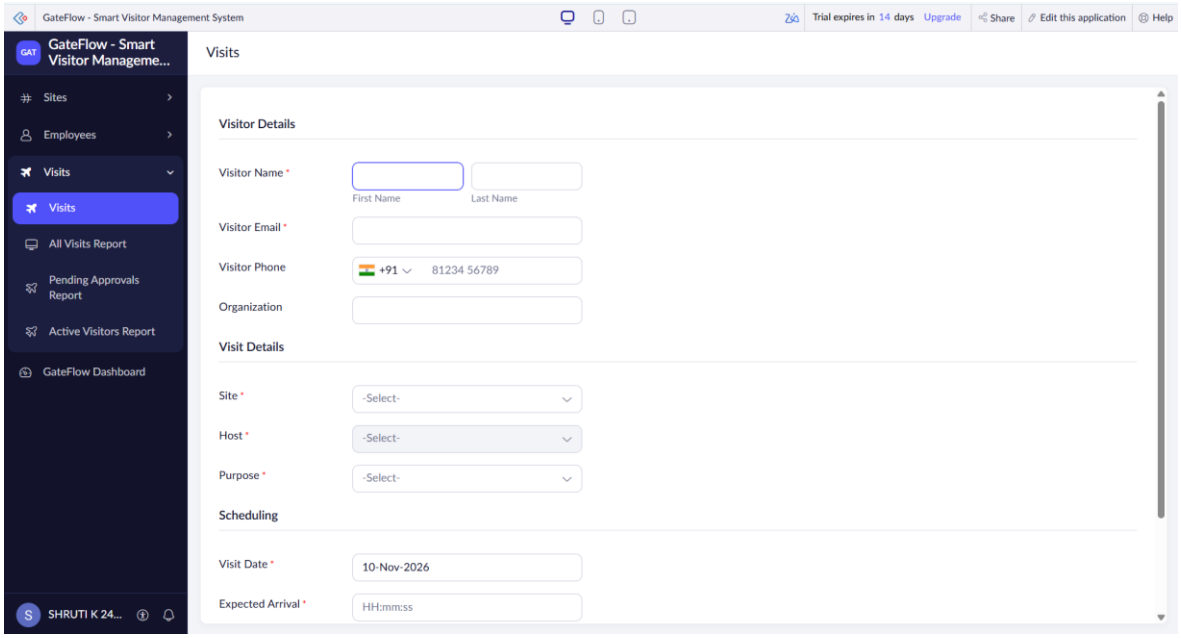


Figure 5.1: Application Home and Navigation Sections

5.2 Forms and Data Capture

Sites form. Stores each location with a unique Site Code, Site Name, an Address field, Maximum Visitors, Timezone (default Asia/Kolkata) and an Is Active checkbox. The Validate Site Capacity workflow allows only the administrator to save a site, requires a positive capacity, and does not allow the capacity of an existing site to be reduced below the number of visitors currently checked in.

Employees form. Stores Full Name, unique Employee Code and Email, Role (default Host), a Site lookup, Is Active and Department. The scripts match Email with zoho.loginuserid to identify the logged-in employee.

Visits form. Has four sections: Visitor Details, Visit Details (Site, Host, Purpose), Scheduling and Internal Processing (Status, Approval Note, Check In/Out Time). On load the internal section is hidden, the status is set to Pending, only active sites are listed and Host stays disabled until a site is chosen. Visitor name, email and phone are marked as personal data (PII).

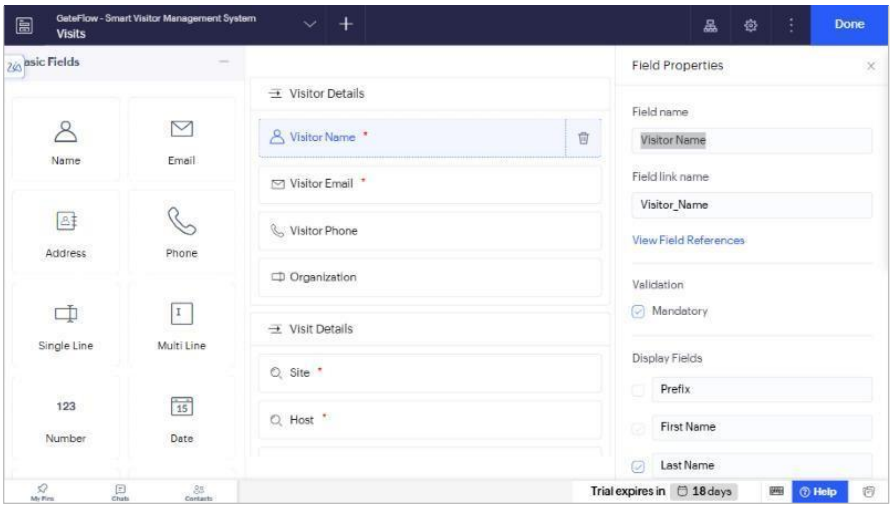
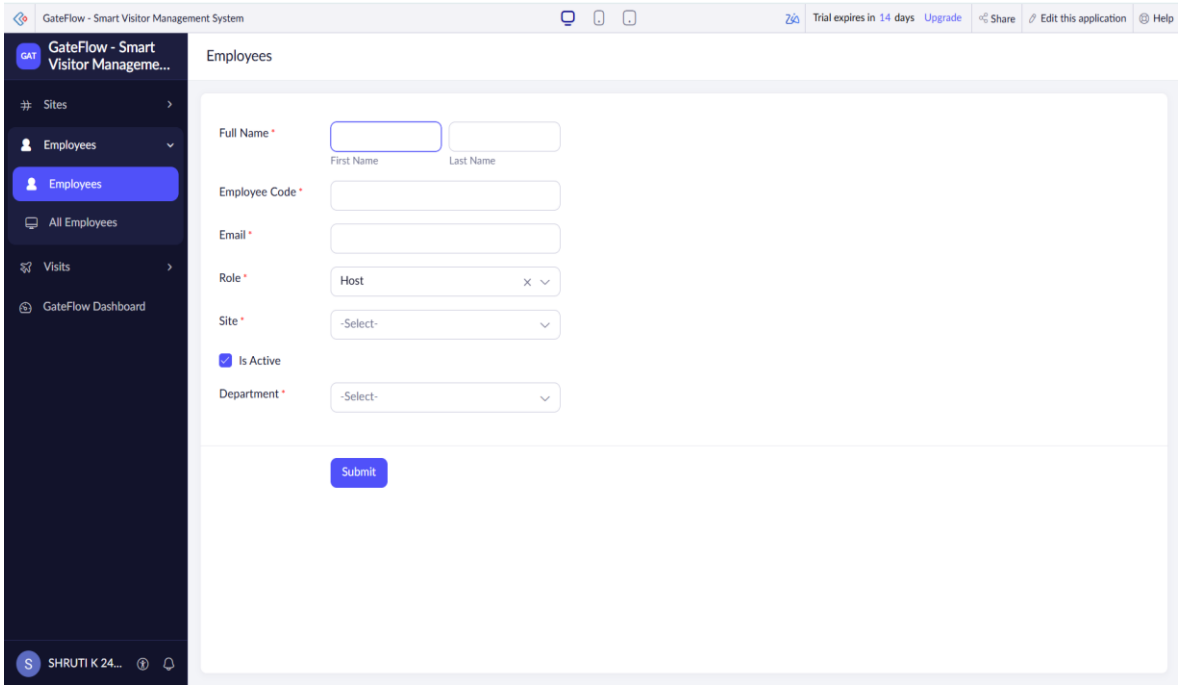


Figure 5.2: Visits Form in the Form Builder



5.3: Visits Form in Live Mode (Host disabled until a Site is selected)

5.3 Reports

Table 5.1: Reports Implemented

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
All Visits Report	List	Visits	Filter: ID in thisapp.VisibleVisitIDs()	Approve, Reject, Check In, Check Out	Admin, Security
Report Name	**Type**	**Source Form**	**Filter / Grouping / Sorting**	**Custom Actions**	**Roles**
Pending Approvals Report	List	Visits	Filter: Status == "Pending" && ID in thisapp.VisibleVisitIDs()	Approve, Reject	Admin, Host
Active Visitors Report	List	Visits	Filter: Status == "Checked In" && ID in thisapp.VisibleVisitIDs()	Check Out	Admin, Security
All Sites	List	Sites	No filter (all sites)	—	Admin
All Employees	List	Employees	No filter (all employees)	—	Admin

All five are list reports; kanban, calendar and pivot views were not needed. The record menu actions all call ProcessVisit.

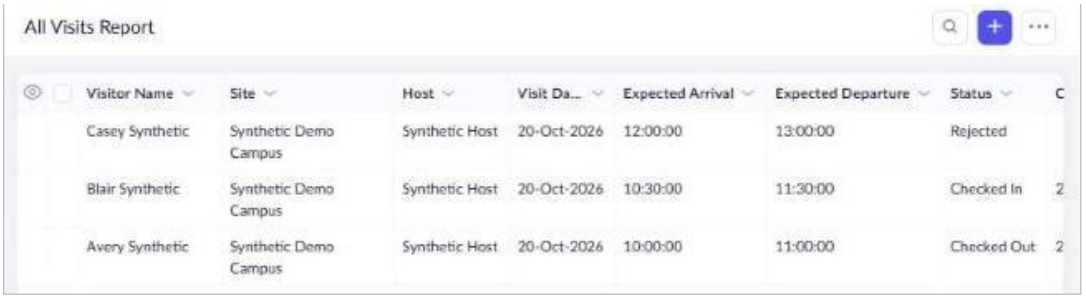


Figure 5.4: All Visits Report with Synthetic Test Records

5.4 Pages and Dashboards

GateFlow_Dashboard is a single Creator page built with the Page Builder. It contains one HTML snippet that mixes Deluge and HTML. The Deluge part first decides which visits the logged-in user may see: all visits for the administrator, visits where the user is the host for a Host, and visits of the user's site for Security. It then loops over those visits and counts Visits Today, Pending Approvals, Currently Inside and Completed Today (visits checked out today). Because this snippet (DS-09) is mostly HTML markup, its logic is described here instead of being listed in Appendix C.

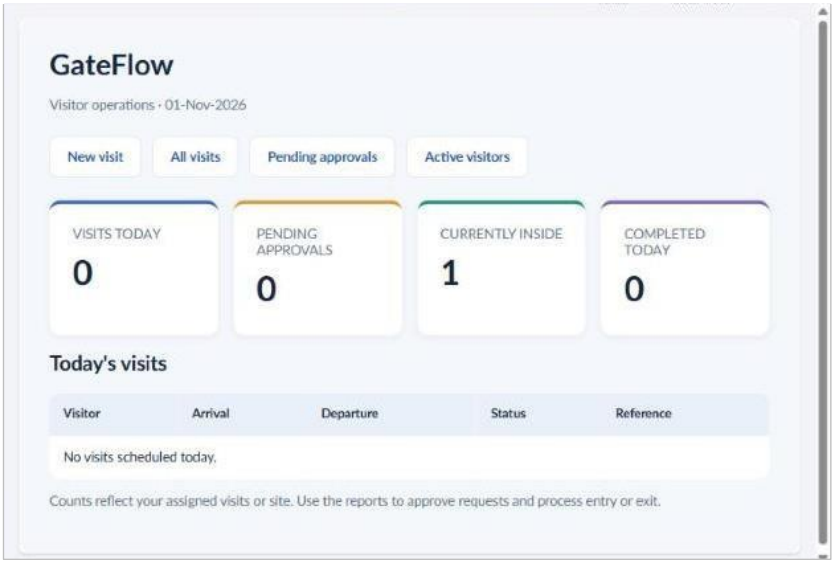


Figure 5.5: GateFlow Dashboard Page (viewed on 01 November 2026)

5.5 Deluge Scripting

Table 5.2: Deluge Scripts Implemented

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-01	Sites → On Validate	Admin-only edits; capacity > 0 and not below occupancy	fetch, count(), cancel submit
DS-02	Visits → On Load	Hide internal fields; defaults; active sites only	hide, clear, ui.add, disable
DS-03	Visits → On User Input (Site)	Load active hosts of the selected site	criteria fetch, getAll()
DS-04	Visits → On Validate	Date/time rules; site and host checks; lock processing fields	if-else, fetch, alert, cancel submit
DS-05	Visits → On Success	Acknowledgement and approval-request emails	sendmail
DS-06	Function ProcessVisit	Approve, reject, check in, check out	function, count(), try-catch, update
DS-07	Function VisibleVisitIDs	Visit IDs a user may see (report filters)	list return, role-based fetch
DS-08	Report actions (4)	Call ProcessVisit and log the result	thisapp call, info
DS-09	Page GateFlow_Dashboard	KPI counts and the list of visits for today	for each, counters

Code 5.1: Date and time checks in Validate Visit Request (DS-04, extract)

```
// DS-04 | Form: Visits | Event: Validations on form submission
if(input.Visit_Date < zoho.currentdate)
{
    alert "Visit Date cannot be in the past.";
    cancel submit;
}
```

Code 5.2: Capacity check inside ProcessVisit (DS-06, extract)

```
// DS-06 | Function: ProcessVisit | action == "Check In"
occupancy = Visits[Site == v.Site && Status == "Checked In"].count();
if(siteRec.Maximum_Visitors == null || siteRec.Maximum_Visitors <= 0
|| occupancy >= siteRec.Maximum_Visitors)
{
    return "Blocked: site visitor capacity has been reached.";
}
v.Check_In_Time=zoho.currenttime;
v.Status="Checked In";
return "Visitor checked in.";
```

DS-04 cancels a submission with a message if the date is in the past, the departure is not after the arrival, or the site or host is inactive or mismatched. New records are forced to Pending; for existing records the status and timestamps cannot be changed by editing, only through the report actions.

DS-06 (Code 5.2) checks that the visit is Approved, dated today and not yet checked in, then counts the site's Checked In visits and blocks the action at Maximum Visitors. With SYN-01 set to one, Blair's check-in was blocked while Avery was inside and succeeded after Avery left. Decision emails are sent inside a try-catch block, so a mail failure does not undo a valid decision.

5.6 Workflows and Automation

Table 5.3: Workflows Implemented

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Validate Site Capacity	Form (Sites)	On validate	—	Block invalid save	DS-01
Prepare Visit Form	Form (Visits)	On load	—	Hide fields, defaults	DS-02
Filter Hosts by Site	Form (Visits)	User input of Site	Site chosen	Load hosts	DS-03
Validate Visit Request	Form (Visits)	On validate	—	Block or set Pending	DS-04

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Notify Visitor and Host	Form (Visits)	On success (create)	—	Two emails	DS-05
Approve / Reject Visit	Report workflow	Action click	In ProcessVisit	Status, note, email	DS-06, 08
Check In / Check Out Visitor	Report workflow	Action click	In ProcessVisit	Time and status	DS-06, 08

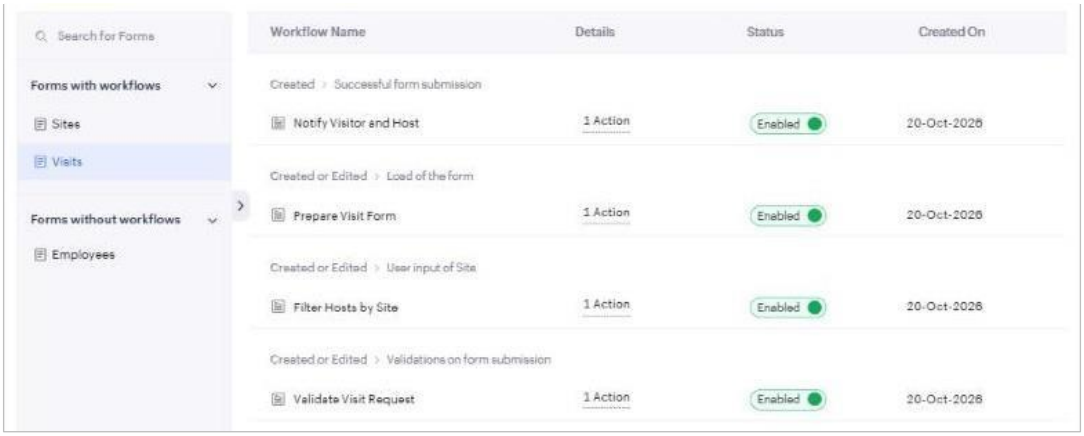


Figure 5.6: Form Workflows Configured on the Visits Form

5.7 Integrations

Table 5.4: Integrations Implemented

Integration	Service / Endpoint	Method	Authentication	Data Exchanged / Purpose
Email notifications	Zoho Mail (Creator sendmail task)	Deluge sendmail in DS-05 and DS-06	Zoho account of the application owner (no connection needed)	Acknowledgement, approval request and approval/rejection emails
REST webhook	Not implemented in this build	—	—	Planned: visit reference, site code and time on check-in

Code 5.3: Acknowledgement email on submission (DS-05, extract)

```
// DS-05 | Form: Visits | Event: Successful form submission
sendmail
[
    from :zoho.adminuserid
    to :input.Visitor_Email
    subject :"GateFlow: visit request received"
    message :"Your visit request for " + input.Visit_Date + " at " +
siteRec.Site_Name + " is Pending. Please wait for the approval email
before arriving. Reference: " + input.ID
]
```



Figure 5.7: Notify Visitor and Host Workflow (sendmail configuration)

5.8 User Management and Role-Based Access

Three roles were created: Admin, with Host and Security under it. The GateFlow Host profile can view only the Pending Approvals Report and the dashboard; GateFlow Security can view two Visits reports and the dashboard; schedules, approval processes and Blueprints were not used. Neither has rights on Sites or Employees, deletion, export or API access. No data sharing rules were added.

Record-level access is enforced by the filters of the three Visits reports, which call VisibleVisitIDs(). The function returns all visit IDs for the administrator, the visits of the matching employee record for a Host and the visits of the employee's site for Security. The Role drop-down on the Employees form is not trusted on its own: ProcessVisit checks it again on the server before any change is made. No customer portal was configured.

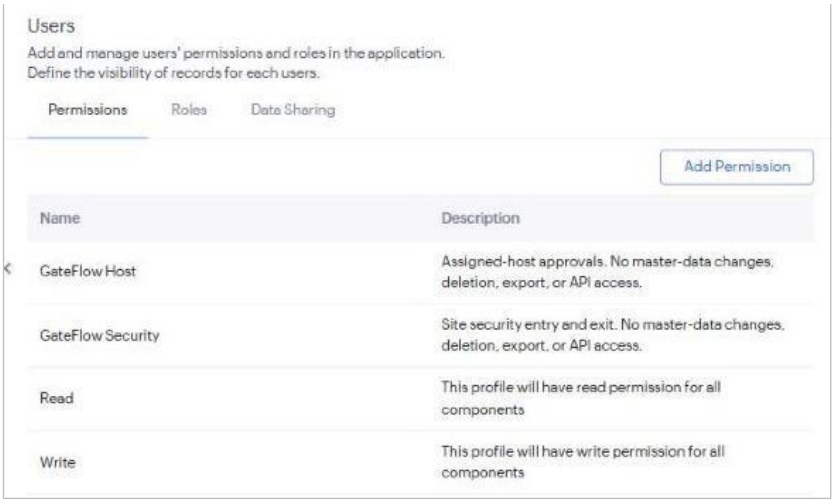


Figure 5.8: Roles and Permission Profiles Configuration (GateFlow Host and GateFlow Security)

5.9 AI and Analytics Features

No Zia feature was used in GateFlow. The application does not contain prediction, OCR, keyword extraction or sentiment fields, and the Zia Build Agent in the builder was not used to generate any part of the application. All forms, workflows, functions and the dashboard snippet were written manually.

5.10 Pro-Code Extension

No pro-code extension was used; all logic runs as Deluge workflows and functions, and the dashboard uses an HTML snippet rather than a JavaScript widget. A Zoho Catalyst function or the Creator REST API could later receive the planned check-in event and pass it to another campus system.

5.11 Mobile and Portal Access

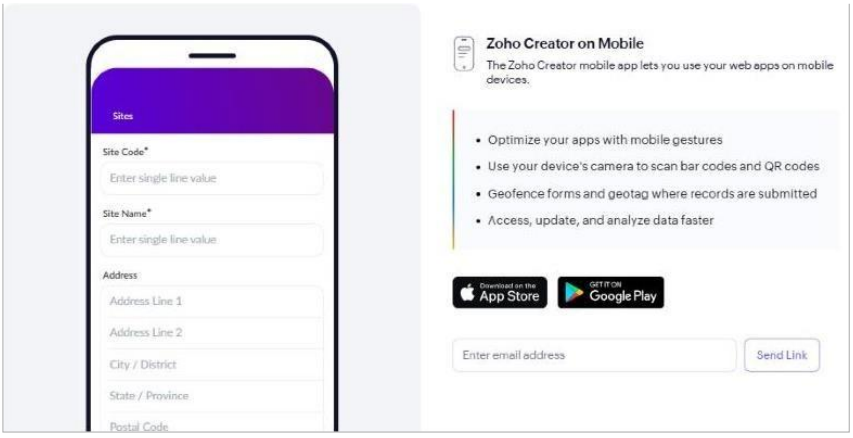


Figure 5.9: GateFlow in the Zoho Creator Tablet Layout Preview

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Testing was done in the Development environment while logged in as the application owner, using clearly marked synthetic data: the site SYN-01 “Synthetic Demo Campus” with a capacity of one, the host SYN-HOST-01 “Synthetic Host” and three visitors named Avery, Blair and Casey Synthetic dated 20 October 2026.

Unit testing covered each form and script separately by entering valid and invalid values. Workflow testing followed one request through submission, approval, check-in and check-out and confirmed the status and time fields after every step. Negative tests repeated actions in the wrong order, for example checking in a pending visit or checking out twice. Finally the reports and the dashboard were compared with the stored records. Separate logins for Host and Security users were not tested, and email delivery to the inbox was not confirmed.

6.2 Test Cases

Table 6.1: Test Cases

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-01	FR-01	Site with capacity 0	Sites form	Blocked	Rejected	Pass
TC-02	FR-04	Past visit date	Date before today	Blocked	Rejected	Pass
TC-03	FR-04	Departure before arrival	11:00 / 10:00	Blocked	Rejected	Pass
TC-04	FR-05	New Visits form	Open form	Host disabled; fields hidden	As expected	Pass
TC-05	FR-03	Three valid requests	Avery, Blair, Casey	Pending	Pending	Pass
TC-06	FR-07	Approve requests	Avery, Blair	Approved	Approved	Pass
TC-07	FR-07	Reject request	Casey	Rejected	Rejected	Pass
TC-08	FR-08	Check in non-approved	Pending visit	Blocked	Blocked	Pass
TC-09	FR-08	Check in within capacity	Avery	Checked In	Checked In	Pass

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-10	FR-08	Check in at capacity	Blair, Avery inside	Blocked	Blocked	Pass
TC-11	FR-09	Check out	Avery	Checked Out	Checked Out	Pass
TC-12	FR-09	Check out twice	Avery again	Blocked	Blocked	Pass
TC-13	FR-08	Slot reuse	Blair	Checked In	Checked In	Pass
TC-14	FR-11, 12	Reports and dashboard	Open both	3 / 0 / 1 / 1	Matched	Pass
TC-15	FR-06	Email delivery	Inbox check	Emails received	Not verified	Open
TC-16	NFR-02	Separate role logins	Host, Security	Scoped access	Not tested	Open

6.3 Debugging Deluge Scripts

Three techniques were used while building the scripts. First, every validation in the form workflows uses an alert followed by cancel submit, so the reason for a rejected submission is shown directly on the form. Second, ProcessVisit never fails silently: each refusal returns a string that starts with “Blocked:” and states the rule that was broken, and each of the four report workflows prints this result with an info statement. This made the capacity test easy to follow, because the blocked check-in for Blair returned the capacity message instead of simply doing nothing.

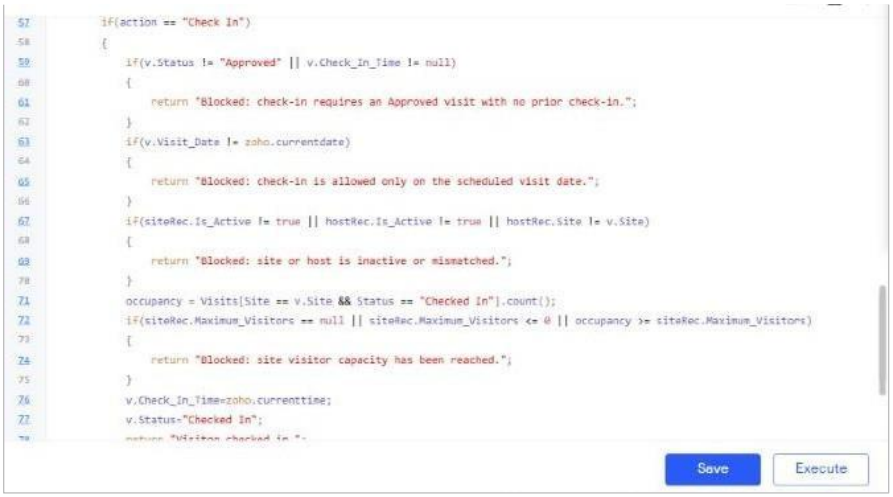


Figure 6.1: ProcessVisit Function in the Deluge Editor (Save / Execute)

6.4 Defect Log

Table 6.2: Defect Log

Bug ID	Description	Root Cause	Fix Applied	Status
BUG-01	Two security users checking in at the same moment could both pass the capacity check.	The occupancy count and the status update are separate statements; no record lock is taken.	Not fixed yet; concurrency test and a locking approach planned.	Open
BUG-02	Delivery of the acknowledgement and decision emails to the inbox is not confirmed.	Only the workflow execution was observed during testing.	Repeat test with a real mailbox.	Open
BUG-03	Dashboard “today” counters show 0 on later dates for the test data.	Synthetic visits are dated 20 October 2026; counters use the current date.	Behaviour is correct; noted for the demo.	Closed

6.5 User Acceptance Testing

Table 6.3: User Acceptance Feedback

User	Role Tested	Feedback	Action Taken
Shruti k	Administrator	Added the site and host, approved requests and handled check-in and check-out; counts matched.	No change needed; reports and dashboard confirmed (TC-14).
Shruti-240701510	Host	Saw only his assigned requests in Pending Approvals; approving and rejecting was quick.	No change needed; assigned-only view confirmed.

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

GateFlow was built and tested only in the Development environment; the Environments page shows nothing in Stage or Production, so it has not been published. The planned path is Stage testing with separate users, removal of synthetic data, then publishing with version notes.

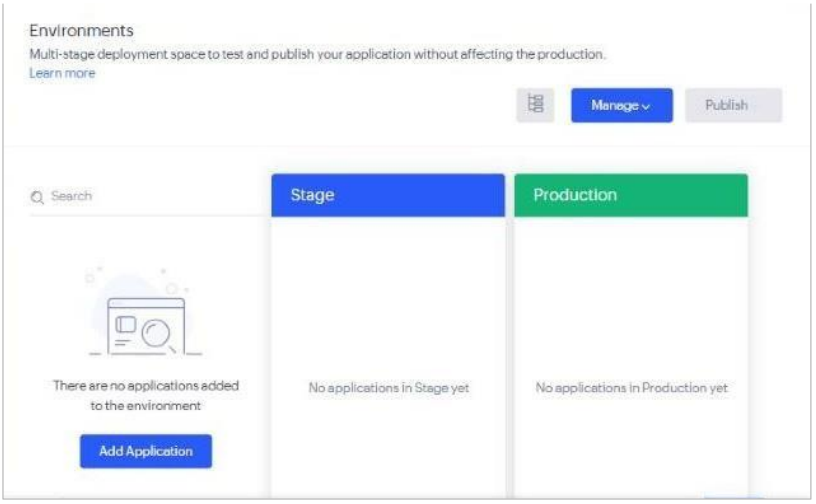


Figure 7.1: Environments Page (Stage and Production not yet used)

Table 7.1: Deployment Checklist

S.No.	Checklist Item	Done (✓)
1	All test cases executed in Development / Stage	✓
2	Sample/test data removed or isolated	✗
3	Roles, permissions and users configured in Production	✗
4	Connections authorised in Production	N/A
5	Schedules and workflows enabled	✓
6	Application published and URL verified	✗
7	Version / release notes recorded	✗

7.2 Performance Optimisation

The scripts keep database calls few: one count() fetch for occupancy, one fetch per lookup, server-side report filters and a single loop for all dashboard counters. Load times were not measured with so few records.

Table 7.2: Performance Optimisation Results

Optimisation Applied	Component	Before	After	Improvement
Single criteria fetch with count() for occupancy	DS-06, DS-01	Not measured	Not measured	One query per check
Report filters applied on the server	All three Visits reports	Not measured	Not measured	Fewer records loaded
One fetch and one loop for all KPI counters	DS-09 Dashboard	Not measured	Not measured	Four counts in one pass

7.3 Security Features

Table 7.3: Security Controls Implemented

Security Control	Where Applied	Threat Mitigated
VisibleVisitIDs() report filter	Three Visits reports	Unauthorised data access
GateFlow Host / Security profiles	Visits reports, dashboard	Master-data change, delete, export
Server-side role and state checks	ProcessVisit (DS-06)	Wrong user or skipped step
Locked and hidden processing fields	DS-02, DS-04	Editing status or times
PII flag, HTML escaping; no stored keys	Personal fields; dashboard	Data exposure; injection

7.4 Governance and Administration

The audit trail under Operations lists every record added to the Visits form with the user and time, for example the test visit added by the owner at 20:10:47 on 20 October 2026. Each decision also stores the approver and time in the Approval Note.

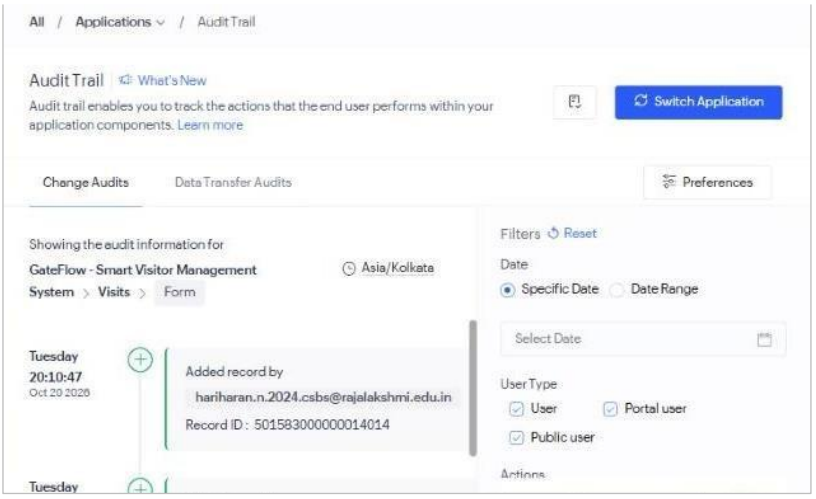


Figure 7.2: Audit Trail of the Visits Form

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

A request for a visit to the Synthetic Demo Campus is entered on the Visits form with the visitor's details, the site and host, the purpose and the scheduled times (Figure 8.1). The record is saved as Pending and appears in the Pending Approvals Report of the host.

The host approves the request. ProcessVisit sets the status to Approved and writes "Approved by shrutikumaran24@gmail.com at 20-Oct-2026 20:12:14" in the Approval Note. On the visit day security checks the visitor in, and the detail view shows the status Checked In with the check-in time 20:12:54 (Figure 8.2). The visitor now appears in the Active Visitors Report, where the Check Out action is available (Figure 8.3), and the dashboard's Currently Inside counter shows 1.

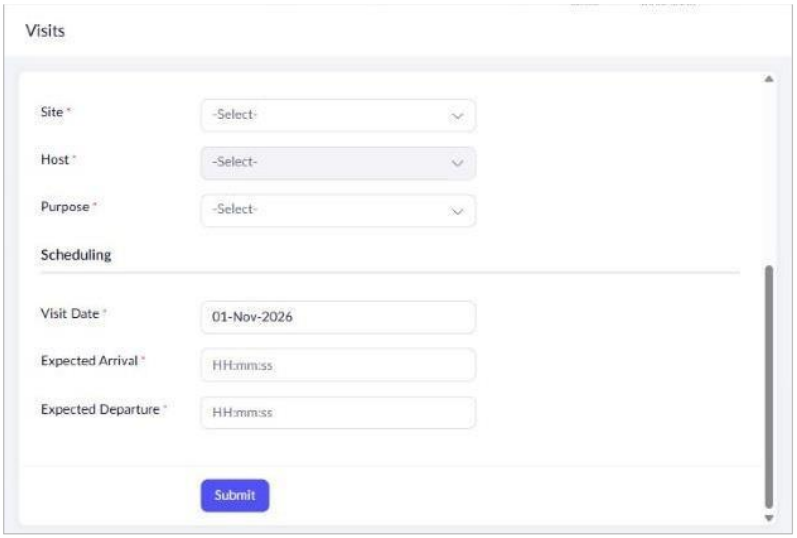


Figure 8.1: Scenario Step 1 – Visit Request entered on the Visits Form

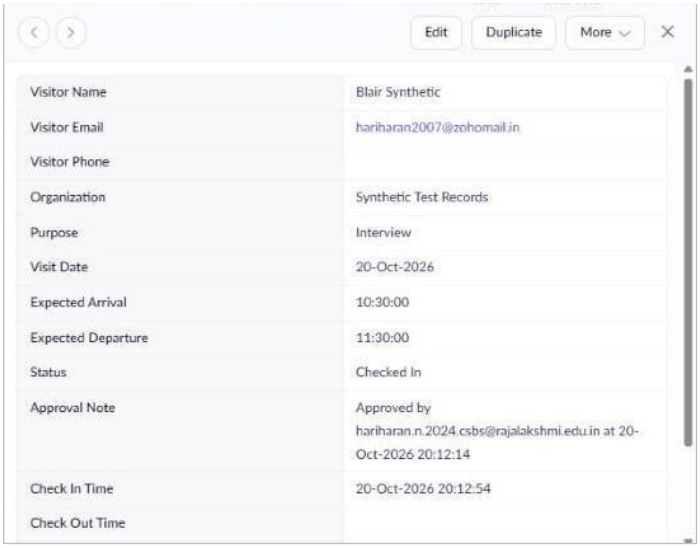


Figure 8.2: Scenario Step 2 – Approved and Checked-In Visit with Approval Note

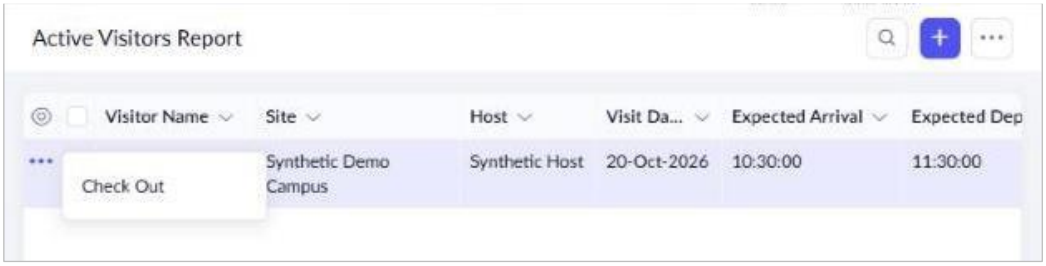


Figure 8.3: Scenario Step 3 – Check Out Action in the Active Visitors Report

8.2 Achievement of Objectives

Table 8.1: Mapping of Objectives to Outcomes

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
Design a normalised data model and forms	Sections 4.2, 4.3, 5.2	Three forms linked by Site and Host lookups (Figures 4.2, 5.2, 5.3)
Automate validation, transitions and capacity control	Sections 5.5, 5.6	Nine workflows and ProcessVisit; TC-02 to TC-13 passed
Provide role-based reports and dashboard	Sections 5.3, 5.4	Three filtered reports and the dashboard; TC-14 matched
Integrate with Zoho Mail	Section 5.7	sendmail in DS-05 and DS-06; inbox delivery still to be verified
Secure the application with roles and filters	Section 5.8, 7.3	Roles, two profiles and VisibleVisitIDs filters; separate logins not yet tested

8.3 Comparison with the Existing System

Table 8.2: Existing System versus Proposed Application

Parameter	Existing System	Proposed Application	Improvement
Time to process one request	Phone call while visitor waits	Approved in advance	Less waiting
Manual steps	Write entry, call host, write times	Click Check In / Check Out	Times set by system
Report generation	Turn register pages	Filtered reports, live counters	Instant view
Data accessibility	One register, open to all at desk	Online, filtered by role	Controlled access

8.4 Limitations of the Low-Code Approach

Creator's built-in approval process could not combine approval with the check-in, check-out and capacity rules, so a single Deluge function and report actions were used instead. Record-level visibility based on the logged-in employee required a custom function in the report filters. The capacity check cannot lock a record, which leaves the concurrency risk noted in BUG-01. Finally, the application, its data and its scripts are tied to the Zoho platform, which is the usual vendor lock-in trade-off.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

Most of GateFlow sits at the no-code end of the spectrum: the three forms, field properties, lookups, the list reports, the navigation menu, roles and permission profiles were configured entirely in visual builders. The low-code part is the Deluge code: the form workflows, the two functions, the four report actions and the dashboard snippet, around 270 lines in total. Nothing in the current build needed pro-code.

Pro-code would be justified for gate hardware events, QR passes at scale, strict locking of simultaneous check-ins or an API for other campus systems.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

GateFlow replaces the paper visitor register with a Zoho Creator application built on three related forms. A visit is requested in advance, checked against date, time, site and host rules, and saved as Pending with email notifications to the visitor and the host. The assigned host approves or rejects it, and security checks the visitor in and out through report actions. One Deluge function controls every status change and enforces the maximum visitor capacity of each site, while role-filtered reports and a dashboard show pending requests and the people currently inside.

9.2 Future Enhancements

- Add the planned HTTPS webhook on check-in, sending only the visit reference, site code and time through a Creator connection.
- Move the application through Stage to Production and test it with separate Host and Security user accounts.
- Add a locking or re-check step so that simultaneous check-ins cannot exceed a site's capacity.
- Add a scheduled workflow that reminds hosts of requests still pending on the morning of the visit.
- Add chart reports, such as visits per site per week, for the administration office.

9.3 Learning Outcomes and Career Relevance

The project gave practical experience of the full low-code cycle: analysing a manual process, designing a relational data model, building forms and reports, writing Deluge for validation and business rules, and setting up roles and record-level security. Writing ProcessVisit in particular showed why business rules should be checked on the server and kept in one place.

REFERENCES

[1] W. Rosam and O. Kalnichevski, *Low-Code Development: A Practical Guide*. New Delhi, India: BPB Publications.

[2] Zoho Corporation, "Zoho Creator Help Documentation." [Online]. Available: https://www.zoho.com/creator/help/ (Accessed: 20 October 2026).

[3] Zoho Corporation, "Deluge Script Reference Guide." [Online]. Available: https://www.zoho.com/deluge/ (Accessed: 20 October 2026).

[4] Zoho Corporation, "Zoho Creator Learning Centre." [Online]. Available: https://www.zoho.com/creator/resources/ (Accessed: 20 October 2026).

[5] Gartner, Inc., "Magic Quadrant for Enterprise Low-Code Application Platforms," *Gartner Research*, 2023.

[6] R. Sahay, A. Indamutsa, D. Di Ruscio and A. Pierantonio, "Supporting the understanding and comparison of low-code development platforms," in *Proc. 46th Euromicro Conf. Software Engineering and Advanced Applications (SEAA)*, 2020, pp. 171–178.

[7] A. C. Bock and U. Frank, "Low-Code Platform," *Business & Information Systems Engineering*, vol. 63, no. 6, pp. 733–740, 2021.

[8] M. Hirzel, "Low-Code Programming Models," *Communications of the ACM*, vol. 66, no. 10, pp. 76–85, 2023.

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Table A.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
I	Application created & sections/menu organised	Y	4 menu sections	5.1	Fig. 5.1
II	Forms with ≥ 8 distinct field types	Y	10+ field types	5.2	Fig. 5.2, 5.3
II	Lookup field (relationship between forms)	Y	Site, Host lookups	4.2, 5.2	Fig. 4.2
II	Subform (parent–child data)	N	Not needed	4.2, 5.2	—
II	Field validations / layout rules	Y	DS-01 to DS-04	5.2	Code 5.1
II	List report with filter, grouping, custom action	Y	Filters, actions	5.3	Fig. 5.4
II	Kanban / Calendar / Timeline / Map report	N	Not used	5.3	—
II	Pivot table / chart report	N	Not used	5.3	—
II	Page / dashboard built with UI Builder	Y	GateFlow_Dashboard	5.4	Fig. 5.5
II	Deluge – form events (validate / success / field actions)	Y	4 events	5.5	Fig. 5.6
II	Deluge – functions, fetch, loops, collections	Y	2 functions	5.5	App. C
II	Email / in-app notifications	Y	sendmail	5.5, 5.6	Code 5.3, Fig. 5.7
II	Form workflow	Y	5 form workflows	5.6	Fig. 5.6
II	Scheduled workflow	N	Not used	5.6	—
II	Approval process	N	Via report actions	5.6	—
II	Blueprint	N	Not used	5.6	—
II	Batch / report workflow	Y	4 report workflows	5.6	Table 5.3
II	Testing & debugging evidence	Y	Chapter 6	6	Fig. 6.1
III	Integration with a Zoho service	Y	Zoho Mail	5.7	Fig. 5.7
III	Connection / REST API call (invokeurl)	N	Not used	5.7	—

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
III	Webhook or Creator REST API	N	Not used	5.7	—
III	Users, roles and permission profiles	Y	3 roles, 2 profiles	5.8	Fig. 5.8
III	Record-level access criteria / customer portal	Y	Report filters	5.8	Table 5.1
III	Zia / AI feature	N	Not used	5.9	—
III	Analytics (charts / Zoho Analytics)	N	Counters only	5.9	Fig. 5.5
IV	Performance optimisation applied & measured	N	Not measured	7.2	Table 7.2
IV	Deployment across environments	N	Dev only	7.1	Fig. 7.1
IV	Security controls (encryption, OAuth, permissions)	Y	Profiles, filters	7.3	Table 7.3
IV	Governance (audit trail, administration)	Y	Audit trail	7.4	Fig. 7.2
V	Pro-code extension (Catalyst / widgets / custom functions)	N	Not used	5.10	—
V	Low-code limitations & spectrum analysis	Y	Sec. 8.4, 8.5	8.4, 8.5	—
—	Mobile app / portal access	N	Preview only	5.11	Fig. 5.9

Total features implemented: 18 / 32

APPENDIX B – COURSE OUTCOME MAPPING

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO1	Explain the principles, architecture, and business value of low-code application development using Zoho Creator.	Ch. 1, 2	3	
CO2	Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting.	Ch. 4, 5 (5.1–5.6), 6	3	
CO3	Integrate Zoho Creator applications with Zoho services and external systems using APIs, connections, and role-based security.	Ch. 4.6, 5.7–5.9	2	
CO4	Deploy, secure, and manage Zoho Creator applications across development and production environments following governance best practices.	Ch. 7	2	
CO5	Evaluate low-code development strategies, AI-assisted application development, and the transition from citizen developer to professional developer.	Ch. 5.9, 5.10, 8.4, 8.5, 9.3	3	

APPENDIX C – COMPLETE DELUGE SOURCE CODE

C.1 Sites – Validate Site Capacity

```
// DS-01 | Sites: On Validate | site rules
if(zoho.loginuserid != zoho.adminuserid)
{
  alert "Only the administrator may manage sites.";
  cancel submit;
}
if(input.Maximum_Visitors <= 0)
{
  alert "Maximum Visitors must be a positive whole number.";
  cancel submit;
}
if(input.ID != null && Visits[Site == input.ID && Status == "Checked
In"].count() > input.Maximum_Visitors)
{
  alert "Capacity cannot be lower than the number of visitors currently
inside.";
  cancel submit;
}
```

C.2 Visits – Prepare Visit Form and Filter Hosts by Site

```
// DS-02 | Visits: On Load | hide fields, defaults
hide Internal_Processing;
hide Status;
hide Approval_Note;
hide Check_In_Time;
hide Check_Out_Time;
if(input.ID == null)
{
  input.Status = "Pending";
  input.Visit_Date = zoho.currentdate;
  clear Site;
  input.Site:ui.add(Sites[Is_Active == true].ID.getAll());
  clear Host;
  disable Host;
}
// DS-03 | Visits: On User Input of Site | host list
clear Host;
if(input.Site != null)
{
  input.Host:ui.add(Employees[Is_Active == true && Site ==
input.Site].ID.getAll());
  enable Host;
}
else
{
  disable Host;
}
```

C.3 Visits – Validate Visit Request and Notify Visitor and Host

```
// DS-04 | Visits: On Validate | request rules
if(input.Visit_Date < zoho.currentdate)
{
  alert "Visit Date cannot be in the past.";
  cancel submit;
}
if(input.Expected_Departure <= input.Expected_Arrival)
{
  alert "Expected Departure must be later than Expected Arrival.";
  cancel submit;
```

```
}
siteRec = Sites[ID == input.Site];
hostRec = Employees[ID == input.Host];
if(siteRec.ID == null || siteRec.Is_Active != true)
{
  alert "Select an active site.";
  cancel submit;
}
if(hostRec.ID == null || hostRec.Is_Active != true || hostRec.Site !=
input.Site)
{
  alert "Host must be active and belong to the selected site.";
  cancel submit;
}
if(input.ID == null)
{
  input.Status = "Pending";
  input.Check_In_Time = null;
  input.Check_Out_Time = null;
  input.Approval_Note = "";
}
else
{
  previous = Visits[ID == input.ID];
  if(input.Status != previous.Status || input.Check_In_Time !=
previous.Check_In_Time || input.Check_Out_Time !=
previous.Check_Out_Time)
  {
    alert "Use the authorized report actions to approve, reject, check in
or check out.";
    cancel submit;
  }
  if(zoho.loginuserid != zoho.adminuserid)
  {
    alert "Only the administrator may edit visit details. Use report
actions for processing.";
    cancel submit;
  }
}
// DS-05 | Visits: On Success | emails
hostRec = Employees[ID == input.Host];
siteRec = Sites[ID == input.Site];
sendmail
[
  from :zoho.adminuserid
  to :input.Visitor_Email
  subject :"GateFlow: visit request received"
  message :"Your visit request for " + input.Visit_Date + " at " +
siteRec.Site_Name + " is Pending. Please wait for the approval email
before arriving. Reference: " + input.ID
]
sendmail
[
  from :zoho.adminuserid
  to :hostRec.Email
  subject :"GateFlow: approval required"
  message :"A visit request for " + input.Visit_Date + " at " +
siteRec.Site_Name + " requires your review. Open GateFlow and use Pending
Approvals. Reference: " + input.ID
]
```

C.4 Functions – ProcessVisit and VisibleVisitIDs (ProcessVisit is called by DS-08)

```
// DS-06 | Function | status changes
string ProcessVisit(int visitId, string action)
{
  v = Visits[ID == visitId];
  if(v.ID == null)
```

```
  {
    return "Blocked: visit not found.";
  }
  actor = Employees[Email == zoho.loginuserid && Is_Active == true];
  isAdmin = zoho.loginuserid == zoho.adminuserid;
  hostRec = Employees[ID == v.Host];
  siteRec = Sites[ID == v.Site];
  if(action == "Approve" || action == "Reject")
  {
    if(!isAdmin && actor.ID == null || actor.ID != v.Host || actor.Role
!= "Host")
    {
      return "Blocked: only the assigned active host or administrator may
decide this visit.";
    }
    if(v.Status != "Pending")
    {
      return "Blocked: only Pending visits can be approved or rejected.";
    }
    if(action == "Approve")
    {
      if(v.Visit_Date < zoho.currentdate || siteRec.Is_Active != true ||
hostRec.Is_Active != true || hostRec.Site != v.Site)
      {
        return "Blocked: visit date, site or host is no longer valid.";
      }
      v.Status="Approved";
      v.Approval_Note="Approved by " + zoho.loginuserid + " at " +
zoho.currenttime;
    }
    else
    {
      v.Status="Rejected";
      v.Approval_Note="Rejected by " + zoho.loginuserid + " at " +
zoho.currenttime;
    }
    try
    {
      sendmail
      [
        from :zoho.adminuserid
        to :v.Visitor_Email
        subject :"GateFlow: visit " + v.Status
        message :"Your visit on " + v.Visit_Date + " at " +
siteRec.Site_Name + " has been " + v.Status + ". Reference: " + v.ID
      ]
    }
    catch (e)
    {
      return v.Status + ". Email could not be sent; contact the
administrator.";
    }
    return "Visit " + v.Status + ".";
  }
  if(action == "Check In" || action == "Check Out")
  {
    if(!isAdmin && actor.ID == null || actor.Role != "Security" ||
actor.Site != v.Site)
    {
      return "Blocked: only security assigned to this site or the
administrator may process entry and exit.";
    }
    if(action == "Check In")
    {
      if(v.Status != "Approved" || v.Check_In_Time != null)
      {
```

```
        return "Blocked: check-in requires an Approved visit with no
prior check-in.";
      }
      if(v.Visit_Date != zoho.currentdate)
      {
        return "Blocked: check-in is allowed only on the scheduled visit
date.";
      }
      if(siteRec.Is_Active != true || hostRec.Is_Active != true ||
hostRec.Site != v.Site)
      {
        return "Blocked: site or host is inactive or mismatched.";
      }
      occupancy = Visits[Site == v.Site && Status == "Checked
In"].count();
      if(siteRec.Maximum_Visitors == null || siteRec.Maximum_Visitors <=
0 || occupancy >= siteRec.Maximum_Visitors)
      {
        return "Blocked: site visitor capacity has been reached.";
      }
      v.Check_In_Time=zoho.currenttime;
      v.Status="Checked In";
      return "Visitor checked in.";
    }
    if(v.Status != "Checked In" || v.Check_In_Time == null ||
v.Check_Out_Time != null)
    {
      return "Blocked: check-out requires a currently checked-in
visitor.";
    }
    v.Check_Out_Time=zoho.currenttime;
    v.Status="Checked Out";
    return "Visitor checked out.";
  }
  return "Blocked: unsupported action.";
}
// DS-07 | Function | visible visit IDs
list VisibleVisitIDs()
{
  if(zoho.loginuserid == zoho.adminuserid)
  {
    return Visits[ID != 0].ID.getAll();
  }
  actor = Employees[Email == zoho.loginuserid && Is_Active == true];
  if(actor.ID != null && actor.Role == "Host")
  {
    return Visits[Host == actor.ID].ID.getAll();
  }
  if(actor.ID != null && actor.Role == "Security")
  {
    return Visits[Site == actor.Site].ID.getAll();
  }
  return List();
}
```

APPENDIX D – USER MANUAL

D.1 Accessing the Application

Open the Gateflow – Intelligent Campus Access and Visitor Authorization Platform using the following URL: https://creatorapp.zoho.in/shrutikumaran24161/gateflow-smart-visitor-management-system. Sign in using the Zoho account that has been granted access to the application. On a desktop, access the application through a web browser, or install the Zoho Creator mobile application on a phone or tablet and sign in using the same account. The application provides role-based access for Admin, Host and Security users. The administrator can manage sites, employees and visitor requests, while hosts can review and approve or reject visit requests. Security personnel can record visitor check-in and check-out. The reports and dashboard help monitor visitor activities and maintain organised campus access records.

D.2 Instructions for Host

- Open Visits → Pending Approvals Report. Only requests where you are the host are listed.
- Click a request to open its details and check the visitor, purpose, date and times.
- Open the record menu (…) and choose Approve or Reject. The visitor receives the decision by email.
- Use the GateFlow Dashboard to see your visits for today and the number of pending requests.

D.3 Instructions for Security

- When a visitor arrives, open Visits → All Visits Report and search for the visitor's name.
- Confirm that the status is Approved and the visit date is today, then choose Check In from the record menu.
- If a "Blocked:" message appears (for example, the site is full), do not admit the visitor and inform the host.
- When the visitor leaves, open the Active Visitors Report and choose Check Out against the visitor.